Report of Locally Linear Embedding

Junyi Li
Sichuan University, China
junyili.cs@gmail.com

1 Mathematical Notations

In this section, we will present all the mathematical notations in the paper and their specific meanings.

1.1 Summary Table of Mathematical Notations

Notation	Type	Corresponding to
$\mathbf{X}_i$	Column vector $(D \times 1)$	The i-th high-dimensional input data sample.
X	Matrix $(D \times N)$	The original high-dimensional data matrix; columns are samples.
$\mathbf{Y}_i$	Column vector $(d \times 1)$	The i-th low-dimensional output (embedding) vector.
Y	Matrix $(d \times N)$	The final low-dimensional embedding matrix; columns are the embedded points.
N	Scalar	The total number of data samples.
D	Scalar	The dimensionality of the original high-dimensional space.
d	Scalar	The dimensionality of the target low-dimensional space ($d \ll D$).
K	Scalar	The number of nearest neighbors for each data point; the main hyperparameter of LLE.
$\mathcal{N}_i$	Set of indices	The set of indices corresponding to the K-nearest neighbors of point $\mathbf{X}_i$.
W_{ij}	Scalar	The reconstruction weight of neighbor $\mathbf{X}_j$ for reconstructing point $\mathbf{X}_i$.
W	Matrix $(N \times N)$	The sparse weight matrix containing all reconstruction weights W_{ij}.
$\mathbf{w}_i$	Column vector $(K \times 1)$	The vector of reconstruction weights for point $\mathbf{X}_i$ from its K neighbors.
$E(W)$	Scalar	The cost function for computing the weights, i.e., the sum of reconstruction errors over all points.
$\Phi(Y)$	Scalar	The cost function for computing the final embedding.
G or S_i	Matrix $(K \times K)$	The local Gram matrix for point $\mathbf{X}_i$, where $(G)_{jk} = (\mathbf{X}_i - \mathbf{X}_j)^T(\mathbf{X}_i - \mathbf{X}_k)$. Used to solve for weights $\mathbf{w}_i$.
M	Matrix $(N \times N)$	The sparse cost matrix for the final embedding step, defined as $\mathbf{M} = (\mathbf{I} - \mathbf{W})^T(\mathbf{I} - \mathbf{W})$. The solution $\mathbf{Y}$ is derived from its eigenvectors.
$\mathbf{I}$	Matrix	Identity matrix.
λ_n	Scalar	The n-th smallest non-zero eigenvalue of the cost matrix M.

Summary of mathematical notations for LLE.

2 Q&A

Next, I will answer the questions one by one.

For the convenience of the subsequent discussion, the entire process of LLE is presented first.

2.1 The LLE Algorithm

Locally Linear Embedding (LLE) is an advanced nonlinear dimensionality reduction technique. Its core lies in the "manifold hypothesis": the idea that high-dimensional data points actually reside on a low-dimensional manifold embedded within the high-dimensional space. The goal of the LLE algorithm is to "unfold" this manifold and find a low-dimensional representation that preserves the geometric structure of the local neighborhoods.

The implementation of the algorithm can be divided into the following three core steps:

1. **Step 1: Select Neighbors**

   For each high-dimensional data point $\mathbf{X}_i$ in the dataset, the algorithm must first identify its "local neighborhood." The most direct method is to find the K data points closest to $\mathbf{X}_i$ in space, typically using the Euclidean distance as the metric.

The parameter K (the number of neighbors) is the most crucial hyperparameter in the LLE algorithm, as its selection directly defines the "local" scope for the subsequent geometric structure analysis.

2. **Step 2: Compute Local Reconstruction Weights**

   After identifying the neighborhoods, the algorithm's goal is to quantify the geometric relationship between each data point and its neighbors. Specifically, the algorithm seeks a set of weight coefficients W_{ij} such that each data point $\mathbf{X}_i$ can be linearly reconstructed from its neighbors $\mathbf{X}_j$ with minimal error. This is achieved by minimizing the following cost function:

$$\mathcal{E}(W) = \sum_i \left\| \mathbf{X}_i - \sum_j W_{ij} \mathbf{X}_j \right\|^2 \tag{1}$$

   When solving for the optimal weights, two key constraints must be enforced:

   - **Sparsity Constraint:** If a data point $\mathbf{X}_j$ does not belong to the K-nearest neighbors of $\mathbf{X}_i$, its corresponding weight W_{ij} must be zero.

$$W_{ij} = 0 \quad \text{if } j \notin \mathcal{N}_i,$$

     where $\mathcal{N}_i$ represents the set of indices for the neighbors of data point $\mathbf{X}_i$. This constraint ensures that the reconstruction of each point is performed **only by its neighbors**, thereby isolating the computation within each local neighborhood.
   - **Invariance Constraint:** For each data point $\mathbf{X}_i$, the sum of the reconstruction weights from all its neighbors must be one.

$$\sum_j W_{ij} = 1 \quad \forall i,$$

     This constraint ensures that the weight combination is independent of the coordinate system, guaranteeing invariance to translations, rotations, and scaling, thus capturing only the intrinsic geometric structure of the neighborhood.

Solving Process Due to the sparsity constraint, the computation of the weight matrix W can be decomposed into N independent optimization problems. For each data point $\mathbf{X}_i$, we need to solve the following constrained minimization problem:

$$\min_{\mathbf{w}_i} \left\| \mathbf{X}_i - \sum_{j \in \mathcal{N}_i} W_{ij} \mathbf{X}_j \right\|^2 \quad \text{s.t.} \quad \sum_{j \in \mathcal{N}_i} W_{ij} = 1,$$

where $\mathbf{w}_i$ is a column vector containing the K weights W_{ij} ($j \in \mathcal{N}_i$) to be found.

First, using the constraint $\sum_j W_{ij} = 1$, we can rewrite the cost function as:

$$\left\| \mathbf{X}_i \sum_j W_{ij} - \sum_j W_{ij} \mathbf{X}_j \right\|^2 = \left\| \sum_j (\mathbf{X}_i - \mathbf{X}_j) W_{ij} \right\|^2.$$

For ease of matrix operations, we define a $D \times K$ matrix $\mathbf{Z}_i$, whose j-th column is $(\mathbf{X}_i - \mathbf{X}_j)$. The cost function can then be expressed as a quadratic form:

$$\|\mathbf{Z}_i \mathbf{w}_i\|^2 = (\mathbf{Z}_i \mathbf{w}_i)^T (\mathbf{Z}_i \mathbf{w}_i) = \mathbf{w}_i^T \mathbf{Z}_i^T \mathbf{Z}_i \mathbf{w}_i.$$

We define the $K \times K$ local covariance matrix as $\mathbf{S}_i = \mathbf{Z}_i^T \mathbf{Z}_i$. The optimization problem thus becomes:

$$\min_{\mathbf{w}_i} \mathbf{w}_i^T \mathbf{S}_i \mathbf{w}_i \quad \text{s.t.} \quad \mathbf{1}_K^T \mathbf{w}_i = 1.$$

This is a classic problem that can be solved using the **method of Lagrange multipliers**. We construct the Lagrangian function $\mathcal{L}$:

$$\mathcal{L}(\mathbf{w}_i, \lambda) = \mathbf{w}_i^T \mathbf{S}_i \mathbf{w}_i + \lambda (\mathbf{1}_K^T \mathbf{w}_i - 1).$$

Taking the partial derivative of $\mathcal{L}$ with respect to $\mathbf{w}_i$ and setting it to zero gives:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_i} = 2\mathbf{S}_i \mathbf{w}_i + \lambda \mathbf{1}_K = 0 \quad \Longrightarrow \quad \mathbf{w}_i = -\frac{\lambda}{2} \mathbf{S}_i^{-1} \mathbf{1}_K.$$

We substitute this result into the constraint $\mathbf{1}_K^T \mathbf{w}_i = 1$ to solve for λ:

$$\mathbf{1}_K^T \left(-\frac{\lambda}{2} \mathbf{S}_i^{-1} \mathbf{1}_K \right) = 1 \quad \Longrightarrow \quad -\frac{\lambda}{2} = \frac{1}{\mathbf{1}_K^T \mathbf{S}_i^{-1} \mathbf{1}_K}.$$

Finally, substituting the expression for $-\frac{\lambda}{2}$ back into the equation for $\mathbf{w}_i$ yields the closed-form solution for the optimal weight vector:

$$\mathbf{w}_i = \frac{\mathbf{S}_i^{-1} \mathbf{1}_K}{\mathbf{1}_K^T \mathbf{S}_i^{-1} \mathbf{1}_K}.$$

By repeating this process for all data points $i = 1, \ldots, N$, we obtain the complete sparse weight matrix W.

3. **Step 3: Compute Low-Dimensional Embedding**

   The final step's objective is to find a corresponding coordinate vector $\mathbf{Y}_i$ in the target low-dimensional space (of dimension d) for all data points, such that the local linear reconstruction relationships established in the high-dimensional space are best preserved. This is achieved by minimizing the following embedding cost function:

$$\min_{\mathbf{Y}} \Phi(\mathbf{Y}) = \min_{\mathbf{Y}} \sum_{i=1}^{N} \left\| \mathbf{Y}_i - \sum_{j=1}^{N} W_{ij} \mathbf{Y}_j \right\|^2 \quad (2)$$

where $\mathbf{Y} = [\mathbf{Y}_1, \mathbf{Y}_2, \ldots, \mathbf{Y}_N]$ is a $d \times N$ matrix containing the low-dimensional coordinates of all points.

To ensure a unique solution, we impose two constraints:

- **Centering Constraint:** $\sum_{i=1}^{N} \mathbf{Y}_i = \mathbf{0}$. This removes the translational freedom of the embedding.
- **Scale/Rotation Constraint:** $\frac{1}{N} \sum_{i=1}^{N} \mathbf{Y}_i \mathbf{Y}_i^T = \mathbf{I}_{d \times d}$. This requires the low-dimensional coordinates to be uncorrelated and have unit variance, fixing the scale and rotation of the embedding.

Solving Process To solve this constrained optimization problem, we first convert the cost function 2 into matrix form. Note that $\mathbf{Y}_i$ can be expressed as $\mathbf{Y}\mathbf{e}_i$, where $\mathbf{e}_i$ is a column vector with a 1 in the i-th position and zeros elsewhere (i.e., the i-th column of the identity matrix $\mathbf{I}_{N \times N}$). The reconstruction term can then be written as:

$$\sum_{j=1}^{N} W_{ij} \mathbf{Y}_j = \mathbf{Y}\mathbf{w}_i,$$

where $\mathbf{w}_i$ is the i-th column of the weight matrix $\mathbf{W}$. Thus, each term in the cost function is:

$$\|\mathbf{Y}_i - \sum_j W_{ij} \mathbf{Y}_j\|^2 = \|\mathbf{Y}\mathbf{e}_i - \mathbf{Y}\mathbf{w}_i\|^2 = \|\mathbf{Y}(\mathbf{e}_i - \mathbf{w}_i)\|^2.$$

Summing over all i, and using the property of the matrix trace that $\sum_i \|\mathbf{A}\mathbf{v}_i\|^2 = \mathrm{Tr}(\mathbf{A}\mathbf{A}^T)$, the cost function can be rewritten as:

$$\Phi(\mathbf{Y}) = \mathrm{Tr}\left(\mathbf{Y}(\mathbf{I} - \mathbf{W})(\mathbf{I} - \mathbf{W})^T \mathbf{Y}^T\right).$$

We define a sparse, symmetric $N \times N$ matrix $\mathbf{M} = (\mathbf{I} - \mathbf{W})^T (\mathbf{I} - \mathbf{W})$. The optimization problem finally becomes:

$$\min_{\mathbf{Y}} \mathrm{Tr}(\mathbf{Y}\mathbf{M}\mathbf{Y}^T) \quad \text{s.t.} \quad \sum \mathbf{Y}_i = \mathbf{0}, \frac{1}{N} \mathbf{Y}\mathbf{Y}^T = \mathbf{I}.$$

According to the Rayleigh-Ritz theorem, the solution to this problem (minimizing the quadratic form $\mathrm{Tr}(\mathbf{Y}\mathbf{M}\mathbf{Y}^T)$) is given by the eigenvectors of the matrix $\mathbf{M}$. Specifically, the **row vectors** of the low-dimensional coordinate matrix $\mathbf{Y}$ should be chosen as the eigenvectors of $\mathbf{M}$ corresponding to its **smallest eigenvalues**.

Selecting the Solution The matrix $\mathbf{M}$ has an important property: because the rows of the weight matrix $\mathbf{W}$ sum to 1, $\mathbf{M}$ is guaranteed to have a smallest eigenvalue of 0, with a corresponding eigenvector of all ones, $\mathbf{1}_N$.

- This zero-eigenvalue solution corresponds to a trivial embedding where all points collapse to the origin. It is uninformative and must be **discarded**. Discarding this solution also naturally satisfies the centering constraint, as all other eigenvectors are orthogonal to $\mathbf{1}_N$.
- Therefore, to obtain a d-dimensional embedding, we should select the eigenvectors corresponding to the **2nd smallest to the (d+1)th smallest eigenvalues** of the matrix $\mathbf{M}$.

These eigenvectors together form the rows of the low-dimensional coordinate matrix $\mathbf{Y}$, thus completing the embedding from the high-dimensional space to the low-dimensional one.

2.2 The Meaning of 'Local' in LLE

The LLE algorithm assumes that the meaningful geometric structure of the data exists primarily within the local neighborhood of each data point, rather than across the global scope of the entire dataset.

The algorithm's goal is to find a set of weights W_{ij} such that each data point $\mathbf{X}_i$ can be best reconstructed by its neighbors $\mathbf{X}_j$. The cost function is as follows:

$$\mathcal{E}(W) = \sum_i \left\| \mathbf{X}_i - \sum_j W_{ij} \mathbf{X}_j \right\|^2$$

At first glance, the outer summation $\sum_i$ in the equation above appears to be a global operation since it iterates over all data points. However, the "locality" of LLE is not determined by the form of the cost function itself, but is instead enforced by a crucial **Sparsity Constraint**:

$$W_{ij} = 0 \quad \text{if } j \notin \mathcal{N}_i.$$

From this, it is clear that each data point $\mathbf{X}_i$ is represented by points from its "local" neighborhood $\mathcal{N}_i$; the contribution from non-local points is zero. This restricts the reconstruction of a point to only receive information from its "local" vicinity.

2.3 The Meaning of 'Linear' in LLE

The "linear" nature of LLE is primarily manifested in the following two aspects:

1. The Assumption of Linear Reconstruction for Local Geometry The underlying assumption of the LLE algorithm is that a curved manifold, on a sufficiently small scale, can be approximated by a linear hyperplane (or "patch"). Mathematically, this means that each data point $\mathbf{X}_i$ can be approximately represented by a **linear combination** of its neighboring points.

This is directly reflected in the algorithm's first cost function:

$$\mathcal{E}(W) = \sum_i \left\| \mathbf{X}_i - \underbrace{\sum_j W_{ij}\mathbf{X}_j}_{\text{Linear combination of neighbors}} \right\|^2$$

The core operation, $\sum_j W_{ij}\mathbf{X}_j$, in the equation above is a standard linear combination. The algorithm's objective is to find an optimal set of linear coefficients W_{ij} for each point $\mathbf{X}_i$, such that it lies as accurately as possible within the local linear subspace spanned by its neighbors. Therefore,"linear" here refers to a **method of local modeling**: using simple linear relationships to describe the local geometric features of a complex nonlinear manifold.

2. Preservation of Linear Relationships after Dimensionality Reduction The second "linear" aspect of LLE is manifested in its dimensionality reduction process. After computing the weight matrix W that describes the local geometry in the high-dimensional space, the algorithm's goal is to find a set of coordinates $\mathbf{Y}$ in the low-dimensional space that can **preserve the exact same linear reconstruction relationships**.

This is reflected in the second cost function:

$$\Phi(Y) = \sum_i \left\| \mathbf{Y}_i - \sum_j W_{ij}\mathbf{Y}_j \right\|^2$$

In this optimization process, the weights W_{ij} are fixed. The low-dimensional coordinates $\mathbf{Y}_i$ that the algorithm seeks must also satisfy the **same linear combination** formed by their neighbors $\mathbf{Y}_j$. The entire process can be understood as taking the local "linear rules" (encoded in W) learned from the high-dimensional space, applying them as hard constraints in the low-dimensional space, and thereby finding a globally optimal layout that satisfies all these rules.

2.4 The Meaning of 'Embedding' in LLE

The term "Embedding" refers to both a **process** and the final **result** of that process. It describes the core task of mapping the original dataset from its high-dimensional observation space to a new, lower-dimensional coordinate system that can reveal its intrinsic structure.

1. "Embedding" as a Goal and Process The goal of LLE is to find a corresponding low-dimensional coordinate vector $\mathbf{Y}_i$ for each high-dimensional data point $\mathbf{X}_i$. The process of finding and constructing this new coordinate system is the "embedding."

In LLE, the guiding principle for this process is explicit: the new low-dimensional coordinate system must **preserve** the local neighborhood geometry learned in the high-dimensional space. This geometric relationship has already been quantified by the weight matrix W. Therefore, the "embedding" process is the process of solving the following optimization problem:

$$\min_Y \Phi(Y) = \min_Y \sum_i \left\| \mathbf{Y}_i - \sum_j W_{ij}\mathbf{Y}_j \right\|^2$$

The equation above is the mathematical core of the entire "embedding" process. It seeks a set of low-dimensional coordinates $\mathbf{Y}$ that minimizes the error between each point $\mathbf{Y}_i$ and its linear reconstruction from its neighbors, $\sum_j W_{ij}\mathbf{Y}_j$. In other words, this process involves rearranging all points in the low-dimensional space until they best satisfy the entire set of local neighborhood rules inherited from the high-dimensional space.

2. "Embedding" as the Final Result The final output of the LLE algorithm—the set of low-dimensional coordinate vectors $\{\mathbf{Y}_1, \mathbf{Y}_2, \ldots, \mathbf{Y}_N\}$ corresponding to the optimal solution of the cost function—is what we call the "embedding." This result is typically a $d \times N$ matrix $\mathbf{Y}$, where d is the target dimension and N is the number of samples.

This "embedding" has a key significance: **it maps high-dimensional data to a low-dimensional space with a compact structure, while preserving their neighborhood relationships,** which facilitates subsequent data processing.

2.5 The Assumptions of LLE

1. The Manifold Hypothesis This is the most fundamental and macroscopic assumption of LLE. It posits that:

- **Data is not randomly scattered:** The observed high-dimensional data points are not randomly filling the high-dimensional space, but are concentrated on or near a **smooth underlying manifold** whose dimension is much lower than the ambient space.

The goal of LLE is precisely to discover and unfold this manifold, revealing the coordinate system formed by these intrinsic parameters.

2. The Local Linearity Assumption This is the direct source of the term "Locally Linear" in the algorithm's name and is the core modeling assumption that distinguishes it from global linear methods. This assumption states that:

- **Globally non-linear, locally linear:** Although the entire data manifold may be highly curved and non-linear globally, on a sufficiently small local scale, its geometry can be well approximated by a linear "patch."

This assumption is the theoretical foundation that allows LLE to reconstruct data points via a linear combination of their neighbors. It also leads to an important practical implication: the choice of the neighborhood parameter K must not be too large, otherwise it would encompass parts of the manifold with high curvature, thus **violating the local linearity assumption**.

3. The Sufficient Sampling Assumption This is a practical assumption about the quality and density of the data, which is crucial for the algorithm's success. It requires that:

- **The manifold is well-sampled:** The sampling of data points must be dense enough so that the local neighborhood of each point can truly reflect the local geometry of the manifold. As the paper states on page 4, the success of the algorithm is "Provided there is sufficient data (such that the manifold is well-sampled)."
- **Neighborhoods overlap:** Sufficient sampling ensures that there is enough overlap between the local neighborhoods of different data points. It is through these overlapping neighborhoods that LLE is able to "stitch" together the local geometric information to construct the global coordinate system of the manifold.

If the data sampling is too sparse, or if there are "holes" in certain regions of the manifold, LLE will be unable to establish reliable local reconstruction relationships, and the quality of the final embedding will be greatly diminished. It may even incorrectly disconnect parts of the manifold that should be connected.

Implicit Requirement: A Connected Graph Although this is more of an algorithmic requirement than a pure data assumption, it is closely related to the preceding assumptions. For LLE to generate a single, globally consistent low-dimensional embedding, the graph formed by all data points through their neighborhood relationships must be **connected**. If the graph is disconnected, the algorithm should be applied separately to each independent connected component, as this usually means the data comes from multiple independent manifolds.

2.6 The Problems to be Solved by LLE

The core problem that LLE is designed to solve is the **Non-linear Dimensionality Reduction of high-dimensional data**.

This problem can be further broken down into the following specific goals and challenges:

1. Uncovering Non-linear Data Structures Many real-world high-dimensional datasets have an intrinsic structure that is not linear. For example, a series of images of a face continuously changing its pose, different styles of handwritten digits, or the conformational changes of a molecule—these data points often form curved, twisted low-dimensional manifolds in the feature space.

- **The core question:** How can one "unfold" this complex non-linear structure and map it to a low-dimensional Euclidean space while preserving the true neighborhood relationships between data points?

Traditional linear methods (like PCA) attempt to capture data variance through a global linear projection, but this fails when dealing with non-linear manifolds, often leading to severe distortions of the manifold structure. LLE is designed to address this limitation by preserving local linear relationships to reconstruct the global non-linear structure.

2. Unsupervised Discovery of Manifold Coordinates LLE solves an "unsupervised" problem, meaning the algorithm does not rely on any external labels or prior knowledge when processing the data.

- **The core question:** Given only the coordinates of high-dimensional data points, how can one automatically discover the fundamental intrinsic degrees of freedom (i.e., the intrinsic coordinates of the manifold) that drive the data's variation?

Through its neighborhood-preserving embedding process, LLE attempts to find a new, low-dimensional coordinate system that reflects the intrinsic geometry of the data. The output low-dimensional coordinates can be seen as a recovery of the "natural" parameterization of the manifold.

3. Effective Data Visualization and Feature Extraction In high-dimensional space, we cannot intuitively understand the distribution and structure of the data. At the same time, high-dimensional features often contain a large amount of redundancy and noise, which can degrade the performance of subsequent machine learning algorithms.

- **The core questions:** How can one generate an intuitive and informative low-dimensional (usually 2D or 3D) visualization for high-dimensional data? How can one extract more concise and effective features for subsequent tasks like classification or clustering?

LLE directly addresses these two problems through its dimensionality reduction capability. The low-dimensional embedding it generates can not only be used directly to create scatter plots for visually inspecting structures like clusters and manifolds, but its low-dimensional coordinates can also serve as a new set of deduplicated features that better reflect the data's essence, thereby improving the efficiency and accuracy of other algorithms.

2.7 The Limitations of LLE

1. Sensitivity to Hyperparameter K The performance of the LLE algorithm is highly dependent on the choice of the number of neighbors, K, which is often its only hyperparameter. The choice of K can be neither too large nor too small:

- **If the value of K is too small:** The local neighborhood may not sufficiently capture the geometric structure of the manifold. More seriously, if the data sampling is not dense enough, a small K value could lead to a disconnected neighborhood graph, making it impossible for the algorithm to form a global embedding.
- **If the value of K is too large:** This directly violates the algorithm's core **local linearity assumption**. An overly large neighborhood will include parts of the manifold with high curvature, making it impossible for a data point to be well-approximated linearly by its neighbors. This leads to large errors in the reconstruction weights, which in turn severely distorts the final embedding result.

How to choose an optimal K value for a specific dataset has no universal rule; it often relies on experience, domain knowledge, or extensive cross-validation experiments, which increases the difficulty of using the algorithm in practice.

2. Sensitivity to Data Quality LLE's performance is closely related to the quality of the data it processes, especially regarding noise and sampling density.

- **Sensitivity to noise:** The second step of LLE—computing local reconstruction weights—is essentially a least-squares fitting problem. It is well known that the least-squares method is very sensitive to noise and outliers. If there is noise in the data, an outlier will not only have a biased reconstruction itself but will also severely affect the weight calculations of other points in its neighborhood, leading to a local propagation of error.
- **High demand for sampling density:** The success of LLE relies on **sufficient and uniform sampling**. If the data sampling on the manifold is sparse or has "holes," the algorithm may not be able to find meaningful local neighborhoods. This could cause it to create erroneous "shortcut" connections between different parts of the manifold or fail to connect the manifold entirely, ultimately destroying the topological structure of the embedding.

3. Topological Instability and Crowding The embedding cost function $\Phi(Y)$ of LLE contains only attractive terms, meaning it only works to pull neighboring points closer in the low-dimensional space, but it has **no repulsive term** to push non-neighboring points apart.

- **Prone to 'crowding':** This characteristic can lead to points that are far apart on the manifold being incorrectly mapped to very close positions in the low-dimensional embedding, as long as their local neighborhood relationships are satisfied. This phenomenon is known as "crowding" or "collapsing."
- **Difficulty with complex topologies:** LLE may encounter difficulties when dealing with data that has a complex topology, such as manifolds with multiple branches, varying densities, or different intrinsic dimensionalities (like the 'barbell' data in Figure 17 of the paper). In such cases, a single value of K is often unable to adapt to all regions, leading to a failed embedding.

2.8 Comparison between PCA and LLE

Although Principal Component Analysis (PCA) and Locally Linear Embedding (LLE) both serve the common purpose of dimensionality reduction, their theoretical foundations, core motivations, and application scenarios are distinctly different. PCA is a global linear method, whereas LLE is a local non-linear method. The specific differences are detailed in Table 1.

Advantages of LLE over PCA

Based on these fundamental differences, LLE demonstrates significant advantages over PCA when dealing with specific types of data:

1. Ability to Handle Non-linear Structures This is the core advantage of LLE. For data that is inherently non-linear (e.g., a "Swiss roll" or an "S" curve), PCA's linear projection will completely fail. It will incorrectly map points that are far apart on the manifold to the same or nearby points in the low-dimensional

Feature	PCA	LLE
Motivation & Core Idea	**Global Variance Maximization**. Assumes that the directions of greatest variance in the data carry the most information. Its goal is to find a global linear projection that maximizes the variance of the projected data.	**Local Neighborhood Preservation**. Assumes that the data is distributed on a non-linear manifold that is locally linear. Its goal is to preserve the local geometric relationships between each data point and its neighbors.
Problem to be Addressed	Aims to find the **linear subspace** that best represents the data. It is ideal for removing linear correlations, data compression, and finding the principal axes of ellipsoidally distributed data.	Aims to "unfold" a **non-linear manifold** embedded in a high-dimensional space. It is specifically designed to discover curved and twisted data structures and to recover their intrinsic low-dimensional coordinates.
Objective Function	**A single global optimization**. Seeks a projection matrix $\mathbf{P}$ to maximize the trace of the covariance matrix of the projected data: $\max_{\mathbf{P}} \text{Tr}(\mathbf{P}^T\mathbf{C}\mathbf{P})$ where $\mathbf{C}$ is the covariance matrix of the original data. The solution is the principal eigenvectors of $\mathbf{C}$.	**A two-stage, local-to-global optimization**. 1. **Local Weights**: Minimize local reconstruction error to find weights $\mathbf{W}$: $\min_{\mathbf{W}} \sum_i \left\|\mathbf{X}_i - \sum_j W_{ij}\mathbf{X}_j\right\|^2$ 2. **Global Embedding**: Use the fixed $\mathbf{W}$ to minimize the embedding error: $\min_{\mathbf{Y}} \sum_i \left\|\mathbf{Y}_i - \sum_j W_{ij}\mathbf{Y}_j\right\|^2$

Table 1: Comparison between PCA and LLE

space, thereby destroying the data's topological structure. In contrast, LLE can identify and "unfold" such non-linear manifolds because it focuses only on preserving local neighborhood relationships, ultimately succeeding in reconstructing the true intrinsic geometry of the data in the low-dimensional space.

2. Weaker Distributional Assumptions The performance of PCA is mathematically tied to the global distribution of the data. When the data follows a multivariate Gaussian distribution, its principal components form a statistically optimal basis. However, for non-Gaussian data, maximizing variance does not necessarily equate to preserving the most important structure. LLE, on the other hand, does not rely on any assumptions about the global distribution of the data. It merely assumes **local linearity**, which is a much weaker and more broadly applicable assumption than global Gaussianity, especially when dealing with complex manifold data from the real world.

3. Focus on Local Structure PCA's objective is global, meaning every point in the dataset (including outliers) affects the calculation of the covariance matrix and thus the final projection directions. In contrast, LLE's weight computation is purely local; the reconstruction of a data point is only influenced by its K neighbors. This makes LLE, to some extent, more robust to isolated outliers that are far from the main body of the manifold, as these outliers will not affect the local weight calculations in other regions of the manifold.

3 Experiment

In this section, we will conduct a series of experiments on the MNIST handwritten digit dataset to evaluate the performance of the Locally Linear Embedding (LLE) algorithm, investigate the influence of its key parameters, and compare it against Principal Component Analysis (PCA).

3.1 Data Introduction

The experimental data used is the same as in the previous PCA report, sourced from the '2k2k.mat' file. This dataset contains 4000 handwritten digit images (0-9) sampled from the MNIST database. Among them, 2000 images serve as the training set, and the other 2000 as the test set. Each image is flattened into a 784-dimensional vector (28×28 pixels).

3.2 Pre-processing

After loading the data, we accurately partition the dataset into a training set (X_train, y_train) and a test set (X_test, y_test) according to the $trainIdx$ and $testIdx$ indices. No other pre-processing steps, such as standardization or normalization, were performed.

3.3 LLE Experiments

The experimental procedure follows these steps:

1. Use LLE to independently reduce the dimensionality of the high-dimensional training and test set features to a specified low-dimensional space.
2. Train a K-Nearest Neighbor classifier (with K=1, i.e., 1-NN) on the dimensionally-reduced training features.
3. Use the trained 1-NN classifier to make predictions on the dimensionally-reduced test set and calculate the classification accuracy.

Parameter Influence Analysis

The LLE algorithm has two core user-specified parameters: the number of neighbors K and the target dimension d. We use the control variable method to explore their impact on model performance.

Influence of the Number of Neighbors K We fix the target dimension $d = 12$ and vary the value of K to observe the change in 1-NN classification accuracy.

K	5	10	15	20	25	30
acc	0.1455	0.2150	**0.2815**	0.2150	0.1190	0.1235

Table 2: 1-NN classification accuracy as a function of the number of neighbors K (with $d = 12$)

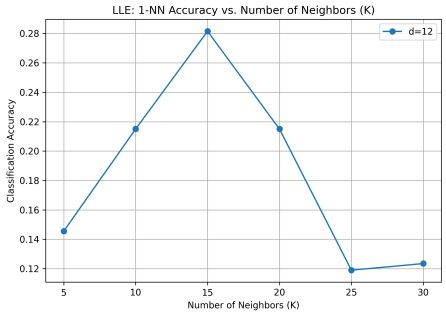


Figure 1: Plot of 1-NN classification accuracy versus the number of neighbors K

The results show that accuracy first increases significantly with K, peaking around $K = 15$. This suggests that when K is too small, the local neighborhood is insufficient to capture the manifold's stable structure; when K is too large, the local linearity assumption may be violated, introducing non-linear errors that lead to a slight decrease in performance.

Influence of the Target Dimension d We fix the number of neighbors at $K = 20$ (a relatively good value) and vary the target dimension d to observe the change in accuracy.

d	2	4	8	12	16
acc	0.1740	0.2775	**0.3130**	0.2150	0.2040

Table 3: 1-NN classification accuracy as a function of the target dimension d (with $K = 20$)

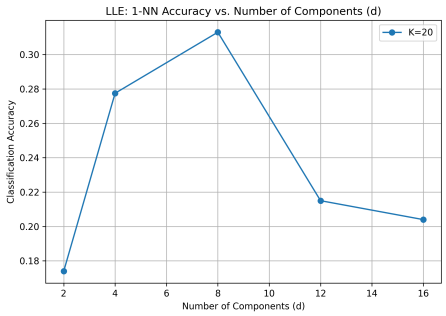


Figure 2: Plot of 1-NN classification accuracy versus the target dimension d

This "rise-then-fall" pattern reveals an important characteristic of the features extracted by LLE. The low-dimensional coordinates computed by LLE correspond to the eigenvectors of the cost matrix $\mathbf{M}$, sorted by their eigenvalues. The first few eigenvectors (excluding the trivial one) with the smallest eigenvalues represent the most stable, global geometric structures of the data, which can be considered the "signal." Subsequent eigenvectors with larger eigenvalues may correspond to more local, unstable, or noise-filled geometric details, which can be considered "noise."

When d exceeds a certain optimal point (in this experiment, 8), we begin to include these "noisy" dimensions as input features for the classifier. Since the 1-NN classifier treats all dimensions equally, these noisy and unstable dimensions can interfere with its decisions, lowering the signal-to-noise ratio and causing a decline in overall classification performance. This explains why performance worsens even as the dimensionality increases.

3.4 Result Analysis

Classification Performance Comparison

For a fair comparison, we select the best-performing parameter combination for LLE ($K = 20, d = 8$) and compare its 1-NN classification performance against PCA, also reduced to 8 dimensions.

Dimensionality Reduction Method (d=8)	1-NN Classification Accuracy
LLE (K=20)	0.3130
PCA	**0.7875**

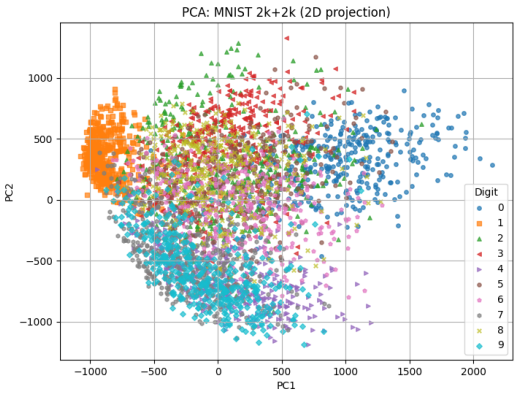
Table 4: Performance comparison between LLE and PCA on the 1-NN classification task.

The experimental results clearly show that PCA achieves a reasonable accuracy of **78.75%**, while LLE's accuracy at its optimal parameters is only **31.30%**, far below PCA and only slightly better than random guessing (10%).

This result seems to contradict LLE's intended purpose of capturing non-linear structures, but its root cause lies in the design of this experiment. As analyzed previously, the experiment required applying LLE **independently** to the training and test sets. This procedure creates two **uncorrelated and unaligned** low-dimensional coordinate systems. Therefore, a 1-NN classifier trained on one coordinate system is completely inapplicable to the other, leading to a sharp decline in classification performance.

In contrast, PCA is an **inductive** method. It learns a single, reusable projection matrix from the training set. When this projection is applied to the test set, it ensures that both datasets are mapped into the **same** low-dimensional space. This allows the classifier to generalize correctly, thereby achieving a reasonable performance.

Therefore, the conclusion from this comparative experiment is not that LLE's features lack discriminative power, but rather a validation that improperly applying a transductive learning method like LLE to an out-of-sample problem will lead to model failure.



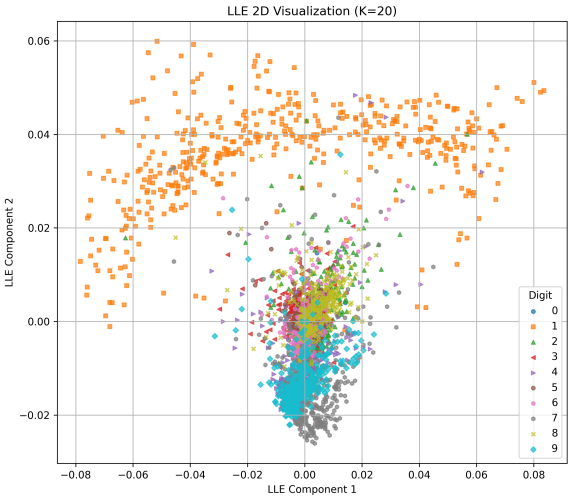


Figure 3: 2D visualization comparison of LLE and PCA on the MNIST dataset.

The visualization results show a significant difference, where LLE's separation of the data is not as clear as PCA's. This leads to a hypothesis: to correctly use LLE for classification, a different strategy must be adopted, such as applying LLE as a preprocessing step to the entire dataset, and then splitting the resulting low-dimensional data into training and test sets.

3.5 Appended Experiment

Given that the previous classification experiment revealed performance issues due to the independent LLE embedding of the training and test sets, we conducted an appended experiment to demonstrate the correct procedure for using LLE for feature extraction and classification.

Corrected Procedure This procedure treats LLE as a **preprocessing step** applied to the entire available dataset to create a unified low-dimensional space.

1. **Merge** the 2000 training samples and 2000 test samples into a single complete dataset of 4000 samples.
2. Apply the LLE algorithm to this complete 4000-sample dataset, reducing its dimensionality from 784 to 8 (using the previously found optimal parameters $K = 20, d = 8$).
3. **Re-partition** the resulting 8-dimensional data back into 2000 training samples and 2000 test samples based on the original indices.
4. On this basis, retrain and re-evaluate the 1-NN classifier.

Comparison of Corrected Results The table below shows the performance of LLE after the corrected procedure and compares it with the previous results.

Dimensionality Reduction Method (d=8)	1-NN Classification Accuracy
PCA	**0.7875**
LLE (Independent Embedding)	0.1980
LLE (Embed then Split)	0.7480

Table 5: Classification performance comparison of LLE (before and after correction) and PCA (d=8)

Analysis and Conclusion The results from the corrected procedure show a significant change and reveal deeper issues:

- **Effectiveness of the Corrected Procedure:** With the correct procedure, LLE's 1-NN classification accuracy surged from the previous 19.80% to **74.80%**. This strongly validates our previous analysis: applying a unified LLE embedding to the entire dataset is a necessary step for its use in subsequent supervised learning tasks.
- **Performance Gap with PCA:** However, it is surprising that even after the correction, LLE's accuracy (74.80%) is still **lower** than PCA's accuracy (78.75%). This indicates that, despite LLE showing superior manifold unfolding in 2D visualization, its generated 8-dimensional features are not more discriminative than PCA's under the current parameters, which may be related to our parameter choices not being optimal.
- **Parameter Sensitivity of LLE:** Unlike PCA, which is parameter-free, the choice of parameters in LLE has a significant impact on the dimensionality reduction of the dataset, making it difficult for users to quickly find a suitable hyperparameter configuration.